

Doing the best at this moment, puts you in the best place for the next moment.



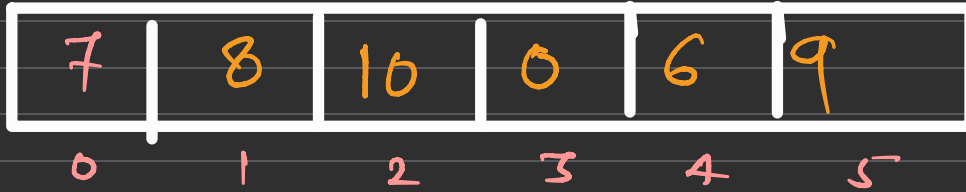
If we have Array, then Why Linked List

index = 1

x = 10

index = 2

arr



12
11

Array
① Static

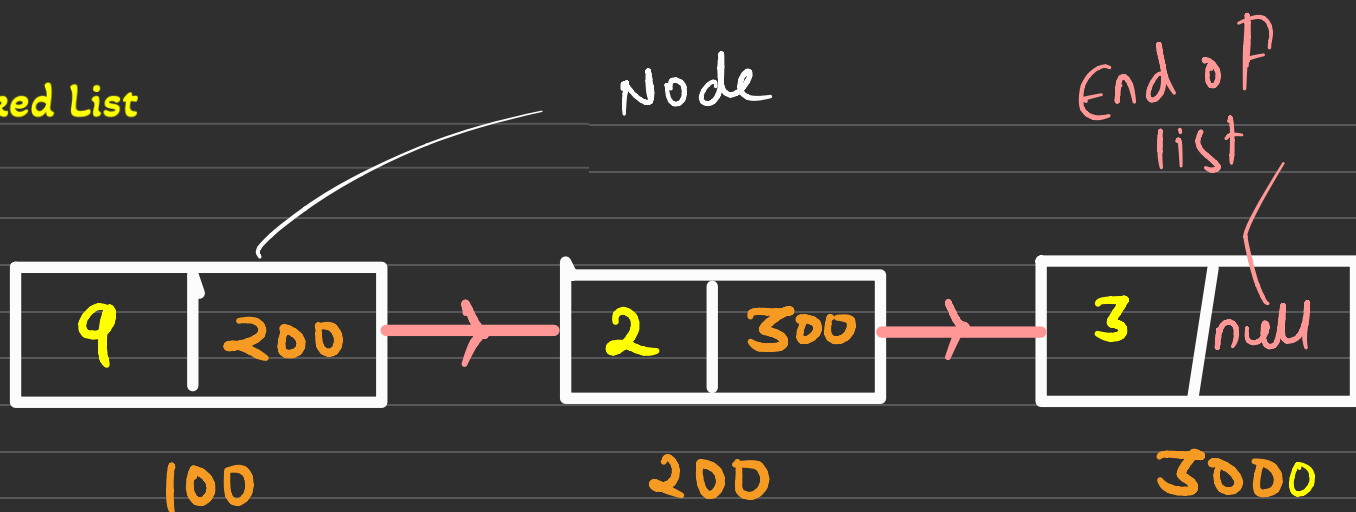
Insert } costly array
delete }

LL
① Dynamic



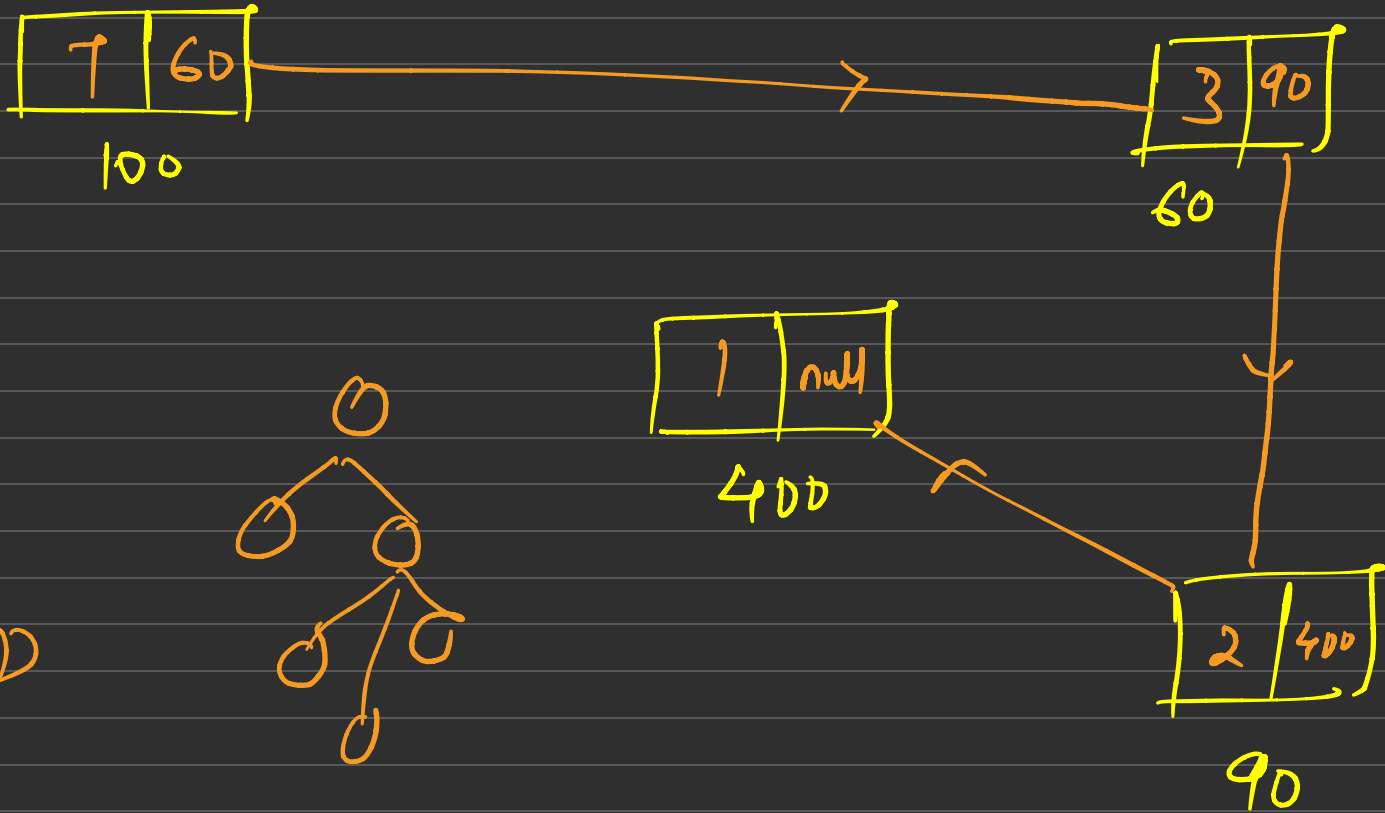
for (i = 0; i < N; i++) (0, 1, 2, ..., (N-1))

What is Linked List



It sequence of elements stored in form Node

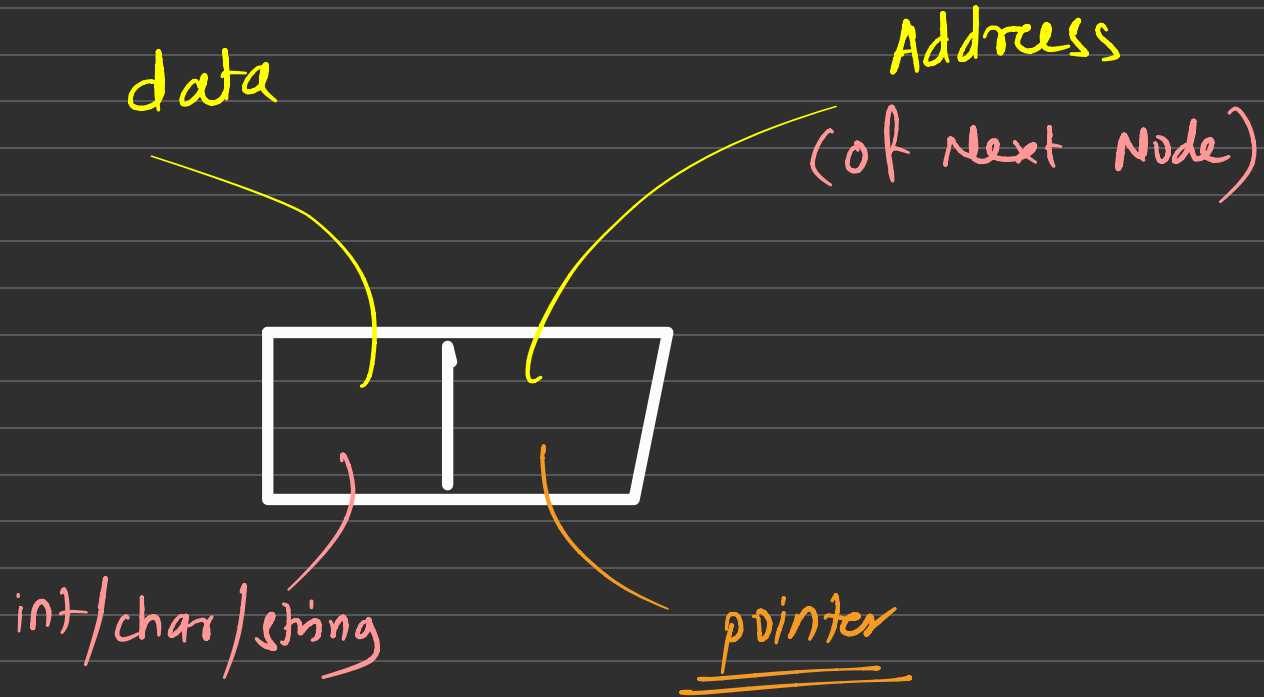
— Linear D.S



LD

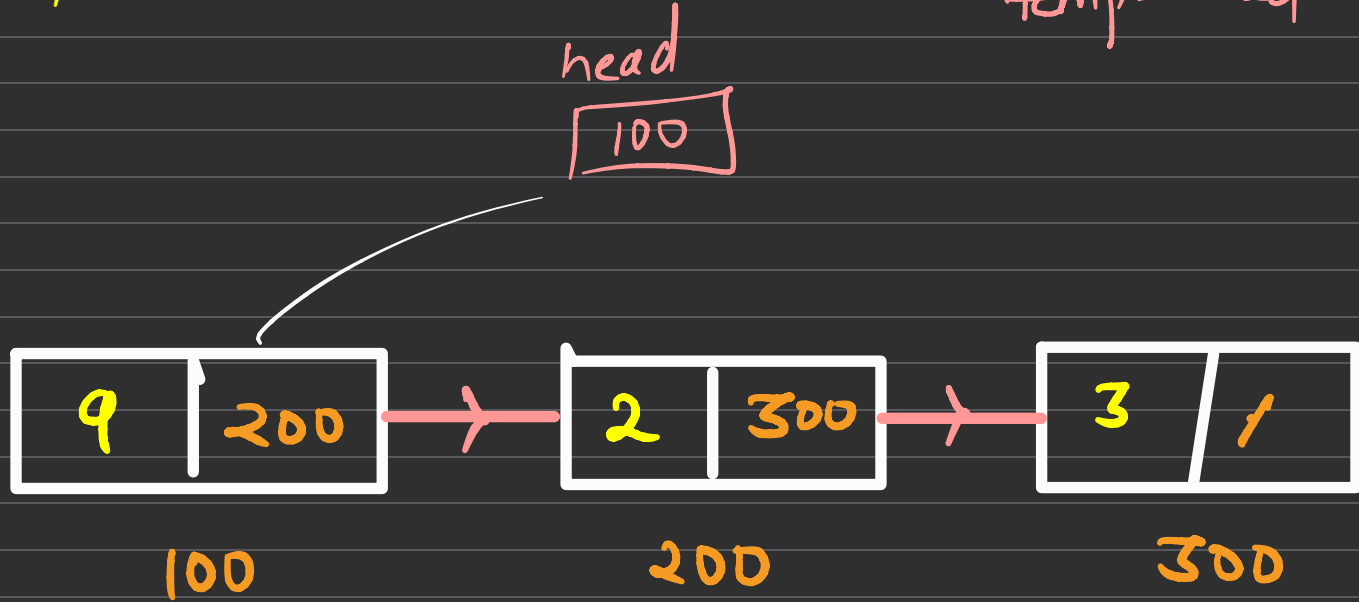
NLD

Node



Head / Start Node

temp = head



→ head == null → Empty list

→ stores the address of First Node

Types of Linked List

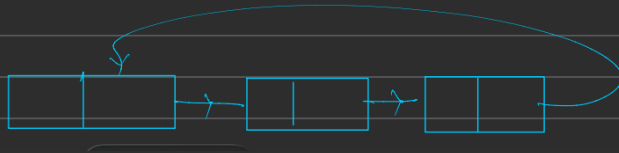
Singly Linked List



Doubly Linked List



Circular Linked List



- Types of operation:

* Insertion

- start
- end
- position

* Traversal

* Deletion

- start
- end
- position

Structure of Linked List (SLL)

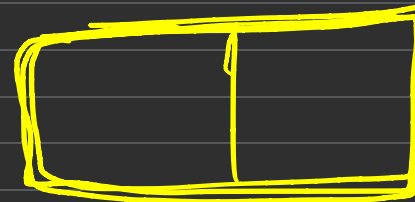
char c
int *p = &c

```
class Node  
{  
    int data;  
    Node * next;
```

```
    Node (int data)  
    {  
        this->data = data  
        next = null  
    }  
};
```



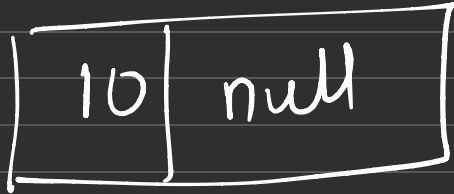
Node



Node

Node *node = new Node(10)

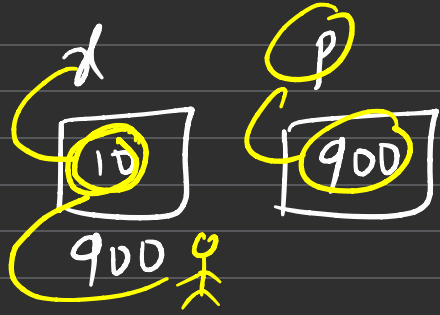
Allocate memory
during Runtime



9000

Obj. member

int x = 10
int *p = &x



temp | 700 |



x → 10
p → 900
*p → 10

java

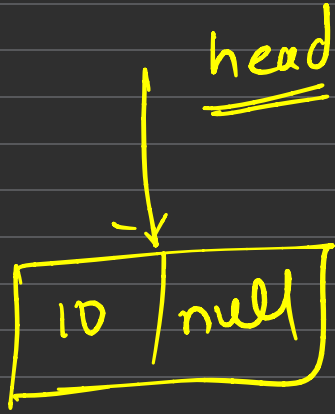
{ temp.data
temp.address

700

CPP

temp → data
(*temp).data

Insert (start)



900
(node)

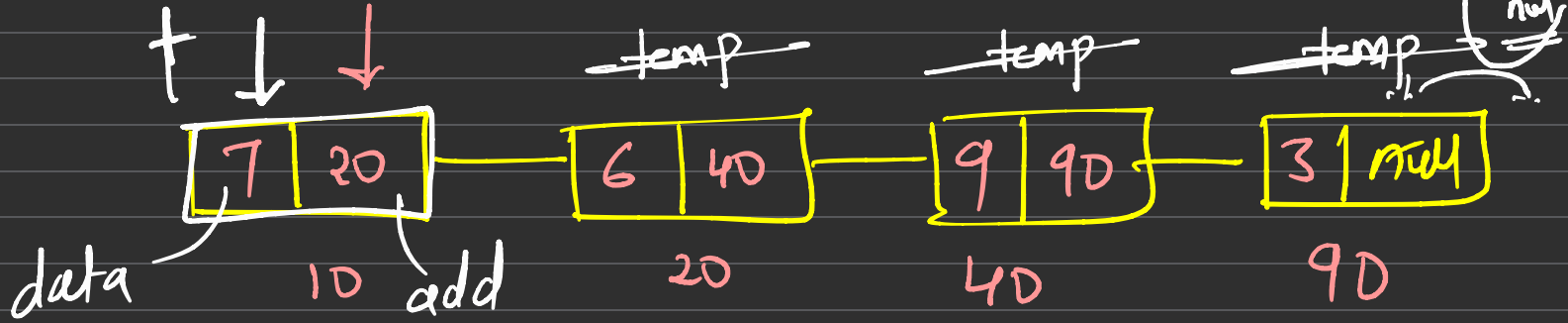
```
if (head == null)
{
    head = node
}
```

```
else
{
    node->next = head
    head = node
}
```

```
}
```

Traverse

head 10



Node * temp = head

```
while ( temp != null )  
{
```

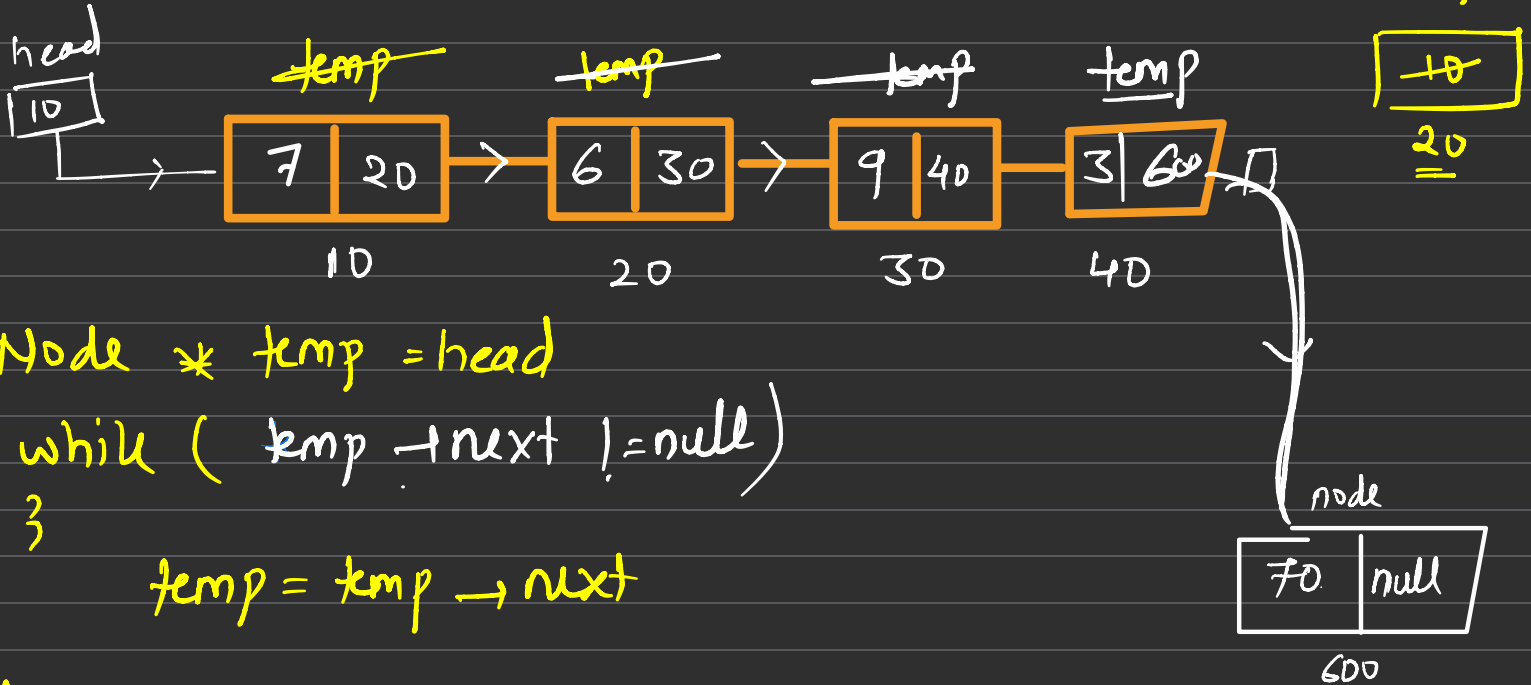
cout << temp->data

temp = temp->next

7 6 9 3

}

Insert at End



Node * temp = head

while (temp → next != null)
{

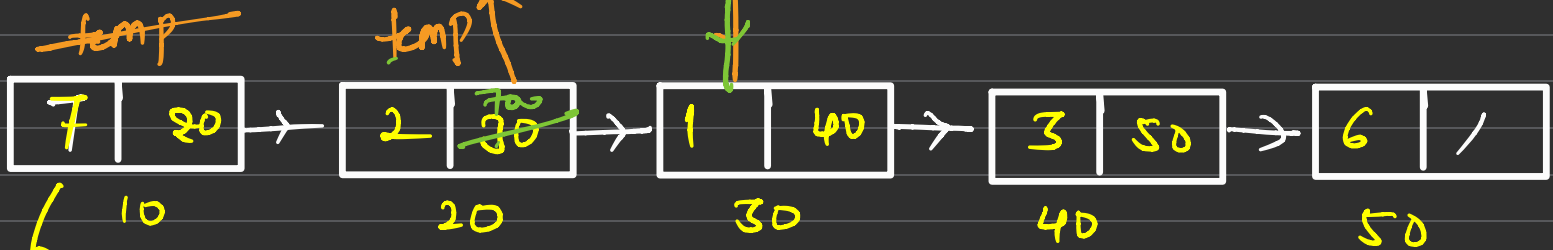
temp = temp → next

}

temp → next = node

Position

$N = 5$



$pos = 3$

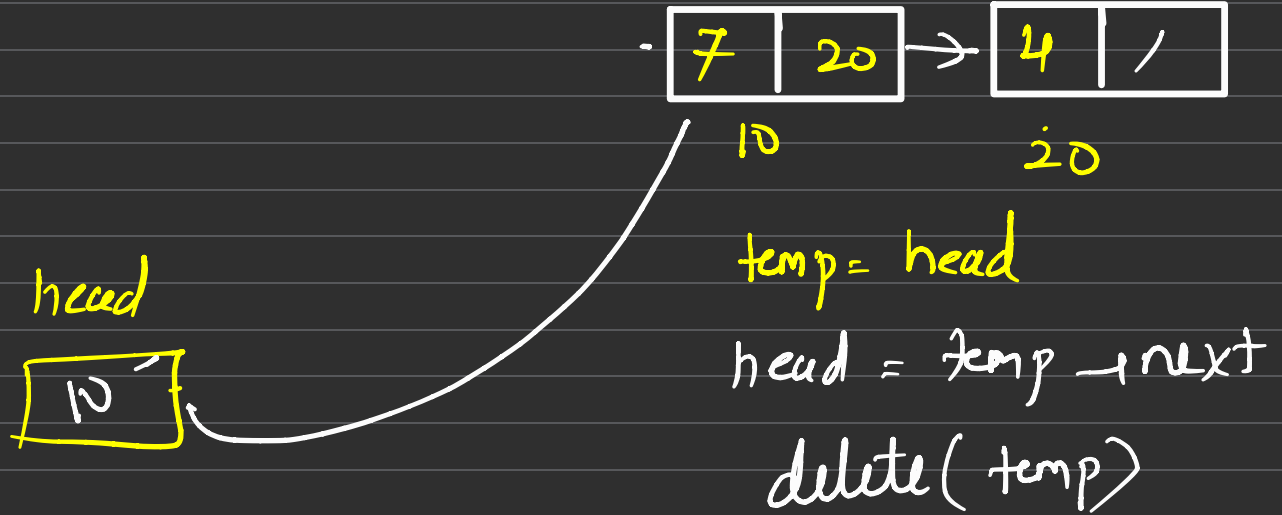
$temp = head$
 $cnt = 1$

if (position > $N+1$)
Invalid

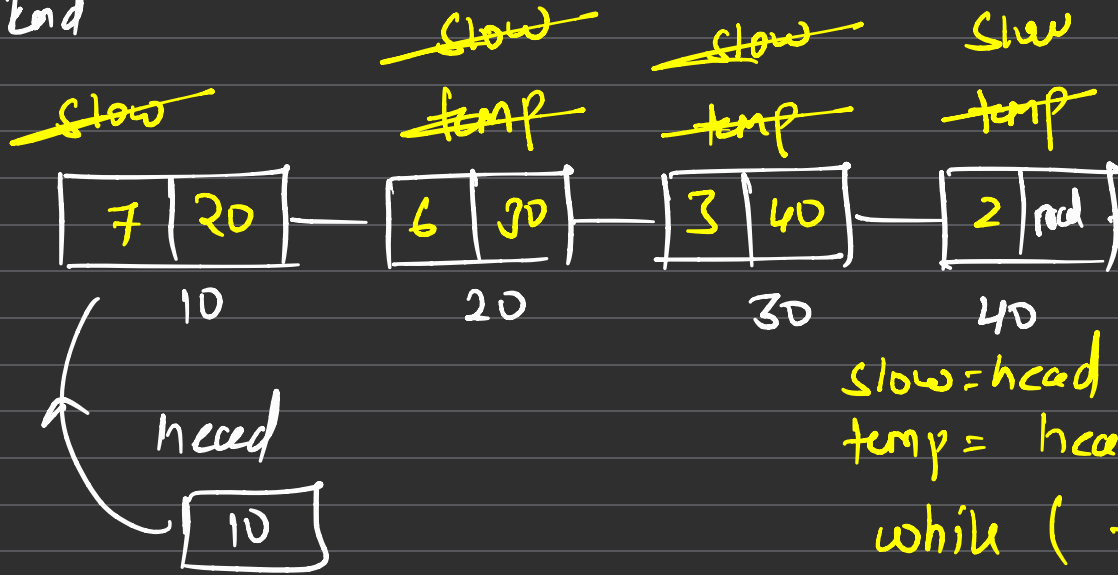
$cnt = 2$

while ($temp \neq null$ && $cnt < pos - 1$)
 $cnt++$
 $temp = temp \rightarrow next$
 $node \rightarrow next = temp \rightarrow next$
 $temp \rightarrow next = node$

Delete at start.



End



slow->next = NULL

delete(temp)

✕